

General Training for
Programming Competitions
& Coding Interviews

Winter Semester 2024/2025

Parameterized Algorithms

Markus Fischer, Paul G. Rump, Christoph Staudt

Friedrich-Schiller-Universität Jena



Parameterized Algorithms

The VertexCover-Problem

Task:

- Given a graph $G = (V, E)$, find a subset of the nodes $W \subseteq V$, such that
- for each edge $e = (u, v) \in E$, at least one of u, v is in W .

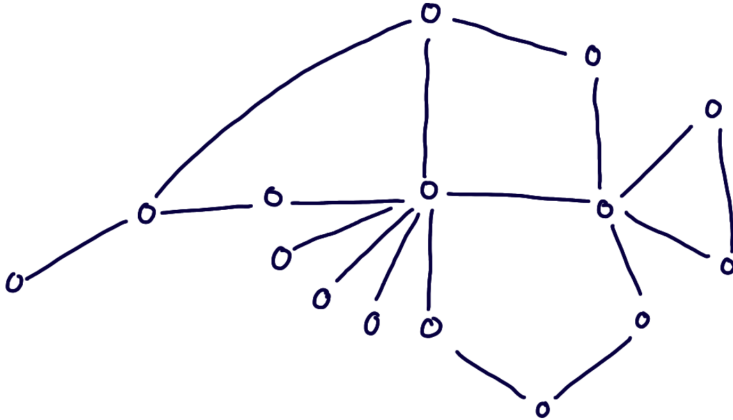
The set W is called a *vertex cover* of G .

If there is no vertex cover $W' \subseteq V$ with $|W'| < |W|$, then W is a *minimal vertex cover*.

The VertexCover-Problem is *NP-hard* (no polynomial-time algorithm is known):

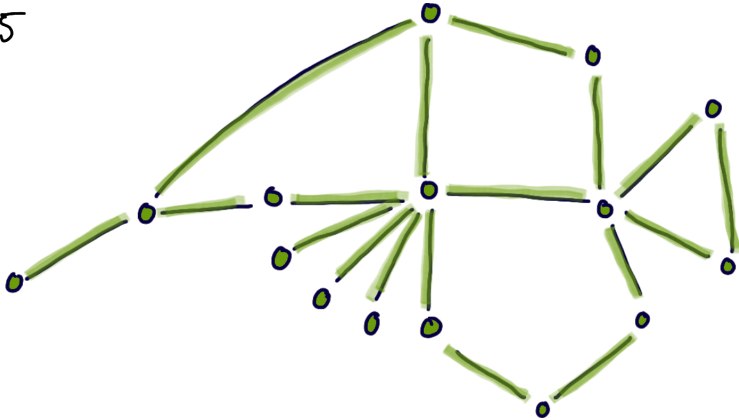
- as optimization problem, that is: “Find a minimal vertex cover”,
- and also as decision problem, that is: “Is there a vertex cover of size k ?”

Vertex-Cover: Examples



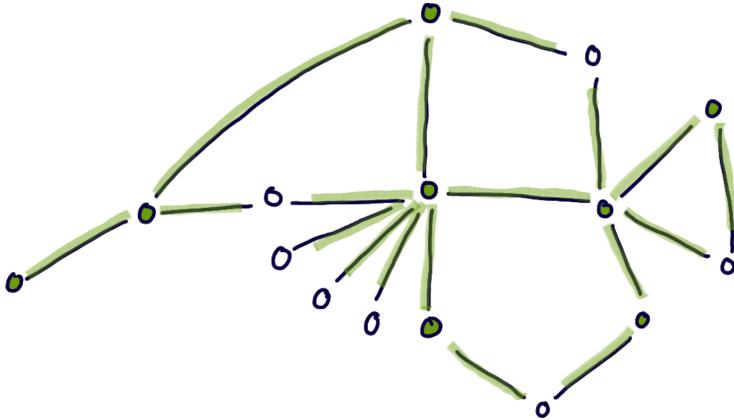
Vertex-Cover: Examples

Vertex Cover
 $k = 15$



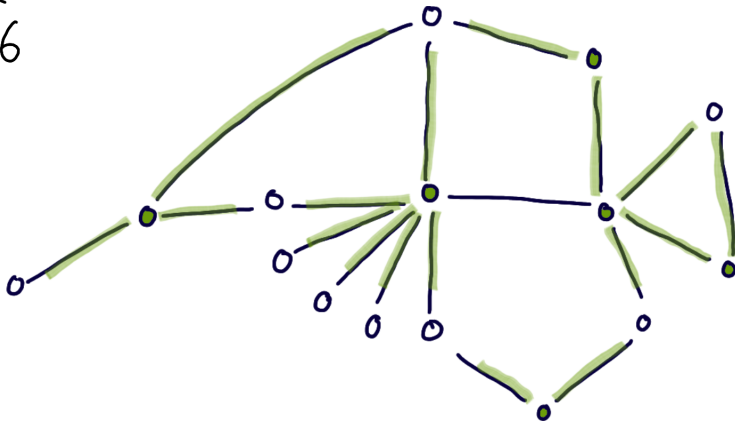
Vertex-Cover: Examples

Vertex Cover
 $k=8$



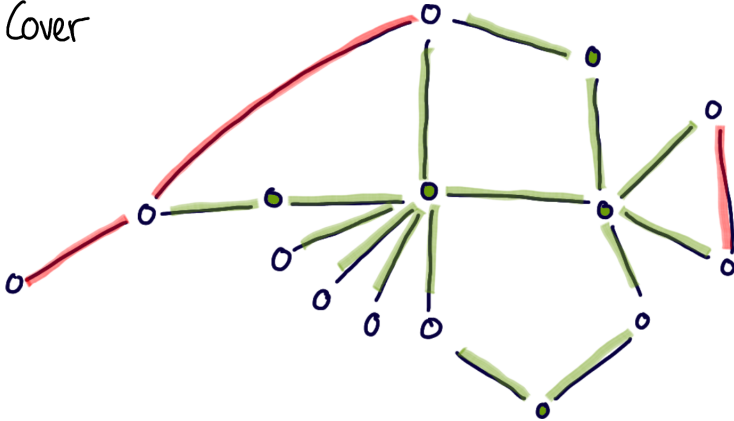
Vertex-Cover: Examples

Vertex Cover
 $k = 6$



Vertex-Cover: Examples

NOT a
Vertex Cover



Applications

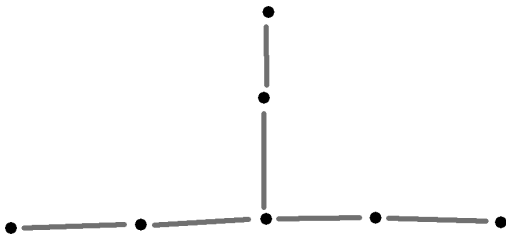
- The “Placing guards in a museum so that all hallways are always guarded”-problem.
- Consider the *Independent Set Problem*: for a graph $G = (V, E)$, find a subset $U \subseteq V$ of the nodes so that no two nodes in U are connected by an edge. Now let $W \subseteq V$ be a vertex cover of G . The set of nodes $U = V \setminus W$ (the nodes that are not part of the vertex cover) are an independent set: no two nodes of U are connected by an edge.

So these problems are related, and one can *find a large independent set* by taking the *complement of a small vertex cover*.

- The Vertex-Cover problem is a special case of the *Set-Cover Problem*: Given a universe $U = \{1, 2, \dots, n\}$ and a set of subsets $S = \{S_i \mid S_i \subseteq U, i = 1, \dots, m\}$, find the smallest subset $S^* \subseteq S$ such that $\bigcup_{s \in S^*} s = U$ (the union of the sets in S^* is the whole universe).

Simple Algorithms for VC

- Full search: check for each subset $W \in V$ if W is a vertex cover, return smallest W (finds minimum vertex cover, but exponential runtime).
- Simple greedy: start with empty W , add any node from $V \setminus W$ until W is a vertex cover ($\mathcal{O}(|G|)$, but no guarantee for minimum).
- Variation of greedy algorithm: always add node with highest degree (still no guarantee for minimum). Example where this greedy variant does not construct a good vertex cover:



Approximation Algorithm for VC

- Better approximation: start with empty W . While W is not a vertex cover, take any uncovered edge and add *both* endpoints to W . This gives a vertex cover that is *at most double the size* of the minimal cover. This is also the *best known* constant-factor approximation algorithm. (PAPADIMITRIOU, Christos H.; STEIGLITZ, Kenneth. Combinatorial optimization: algorithms and complexity. Courier Corporation, 1998.)

k-VertexCover

The k-VertexCover-Problem is a variation of the VertexCover Problem that asks if a graph has a vertex cover of size *at most* k .

The additional parameter can be used when designing algorithms, because we do not have to consider settings that would lead to a vertex cover of size $> k$.

One possible algorithm is a *Search Tree Algorithm* that searches through all possible solutions. In general, this would take exponentially many steps. But by omitting solutions that would lead to a vertex cover of size $> k$, this approach becomes feasible.

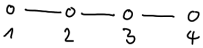
Parameterized Algorithm for k-VertexCover

A *Search Tree Algorithm* that stores intermediate steps to search through all possible solutions in a structured way.

Each node of the search tree contains a pair (H, S) , where $H \subseteq G$, and $S \subseteq V$ with $|S| \leq k$. S is the current vertex cover, and H is the remaining non-covered graph.

1. We initialize the tree with the root node $(G, \emptyset)$.
2. While there is a leaf node (H, S) with $E(H) \neq \emptyset$ and $|S| < k$,
 - 2.1 we choose an arbitrary edge (u, v) from H and
 - 2.2 add the nodes $(H - u, S \cup \{u\})$ and $(H - v, S \cup \{v\})$ as children to (H, S) .

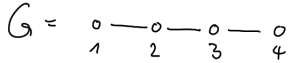
Search-Tree Algorithm for k-VertexCover: Example

$G =$ 

$k=2$



Search-Tree Algorithm for k-VertexCover: Example



$k=2$

$(G, \emptyset)$

Search-Tree Algorithm for k-VertexCover: Example

$$G = \begin{array}{cccc} \circ & - & \circ & - & \circ & - & \circ \\ v_1 & & v_2 & & v_3 & & v_4 \end{array}$$

$$k=2$$

1) choose edge $\{v_1, v_2\}$

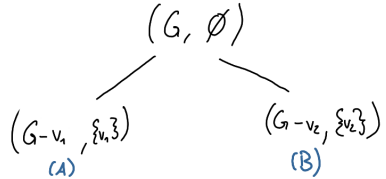
$$(G, \emptyset)$$

Search-Tree Algorithm for k-VertexCover: Example

$$G = \begin{array}{cccc} \circ & - & \circ & - & \circ & - & \circ \\ v_1 & & v_2 & & v_3 & & v_4 \end{array}$$

$$k=2$$

1) choose edge $\{v_1, v_2\}$



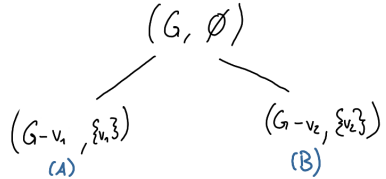
Search-Tree Algorithm for k-VertexCover: Example

$$G = \begin{array}{cccc} \circ & - & \circ & - & \circ & - & \circ \\ v_1 & & v_2 & & v_3 & & v_4 \end{array}$$

$$k=2$$

1) choose edge $\{v_1, v_2\}$

2) choose node (A) and edge $\{v_2, v_3\}$



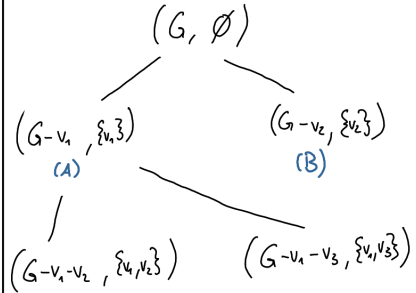
Search-Tree Algorithm for k-VertexCover: Example

$$G = \begin{array}{cccc} \circ & - & \circ & - & \circ & - & \circ \\ v_1 & & v_2 & & v_3 & & v_4 \end{array}$$

$$k=2$$

1) choose edge $\{v_1, v_2\}$

2) choose node (A) and edge $\{v_2, v_3\}$



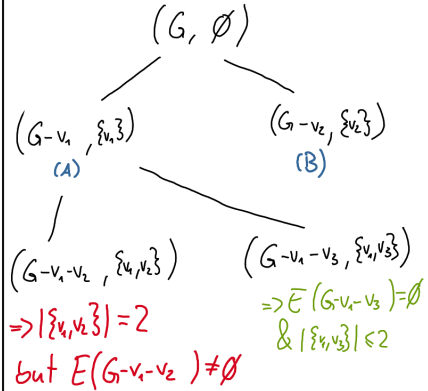
Search-Tree Algorithm for k-VertexCover: Example

$$G = \begin{array}{cccc} \circ & - & \circ & - & \circ & - & \circ \\ v_1 & & v_2 & & v_3 & & v_4 \end{array}$$

$$k=2$$

1) choose edge $\{v_1, v_2\}$

2) choose node (A) and edge $\{v_2, v_3\}$



Parameterized Algorithm for k-VertexCover

The algorithmic complexity of this algorithm depends on two factors:

- the size of the search tree: $\mathcal{O}(2^k)$ (binary tree of depth k) and
- the effort to add the two child-nodes: $\mathcal{O}(|G|)$ for modifying the graph (probably less in practice).

Therefore the total complexity is in $\mathcal{O}(2^k \cdot n^2)$
(where n is the number of nodes in G).

Note that this is *only exponential in k* . Therefore for “small k ” (that is: $k \in \mathcal{O}(\log n)$), this is an exact algorithm for k-VertexCover that is feasible in practice.

An Improved Parameterized Algorithm for k-VertexCover

- Observation: for each edge, one of its nodes has to be in the vertex cover.
- When considering nodes instead of edges, this implies: *either the node* has to be in the vertex cover, *or all of its neighbors*.
- Consider the following variation of the search-tree-algorithm: when adding new leaves, we now consider nodes (with degree > 2) instead of edges. In particular, given a tree-node (H, S) :
 - choose an arbitrary node $v \in V(H)$ (with $\text{degree}(v) > 2$)
 - add the nodes $(H - v, S \cup \{v\})$ and $(H - N(v), S \cup N(v))$ as children of (H, S) .
- Afterwards, there might be leaves (H, S) where H is not empty (can contain nodes of degree ≤ 2). However, we can find a (minimal) vertex cover for this graph H in polynomial time.

An Improved Parameterized Algorithm for k-VertexCover

This improved algorithm builds an asymmetric search tree, where the child-trees of a node at level k have different sizes: $T(k) \leq T(k-1) + T(k-2)$.

(Here the *level* is the number of nodes that can still be added to the vertex cover.)

With the assumption that the tree-sizes are exponential (i. e. $T(k) = c^k$ for some $c > 0$), we get:

$$\begin{aligned}c^k &= c^{k-1} + c^{k-2} \\ \Leftrightarrow c^2 &= c + 1 \\ \Leftrightarrow c^2 - c - 1 &= 0 \\ \Rightarrow c &= \frac{1 + \sqrt{5}}{2} \approx 1.6181 < 1.7\end{aligned}$$

Therefore the runtime of this algorithm is in $\mathcal{O}(1.7^k \cdot |G|^c)$.

Interlude: Parameterized Complexity

Let $L \subset (\Sigma^* \times \mathbb{N})$ be a decision problem.

For example: L can be the set of all tuples (G, k) , where G is a graph that has a vertex cover of size k .

L is called *fixed-parameter tractable (FPT)* if an algorithm exists that can decide $(x, k) \in L$ for any input (x, k) in time $\mathcal{O}(f(k) \cdot |x|^c)$, where f is some *computable* function that does not depend on x , and $c \in \mathbb{R}$ is a constant (that does not depend on x or k).

Parameterization allows us to *distinguish between “easy” and “hard” instances* of NP-hard problems.

Interlude: Parameterized Complexity

Examples for the Complexity Class FPT:

- VertexCover is in FPT.
- The complexity class P is a subset of FPT.

Interlude: Parameterized Complexity

A well-known counter example is the parameterized version of the Clique-problem:
Given a graph G and an integer k , decide if G has a clique of size k .

But checking if a graph contains a clique of size k takes $\mathcal{O}(n^k)$ time. This complexity is only exponential in k , but also depends on n (cannot be written as $f(k) \cdot n^c$ for constant c).
Therefore this problem is (most likely) **not** in $\text{FPT} = \text{W}(0)$.

Problem Kernels & Preprocessing

Problem Kernels: Motivation

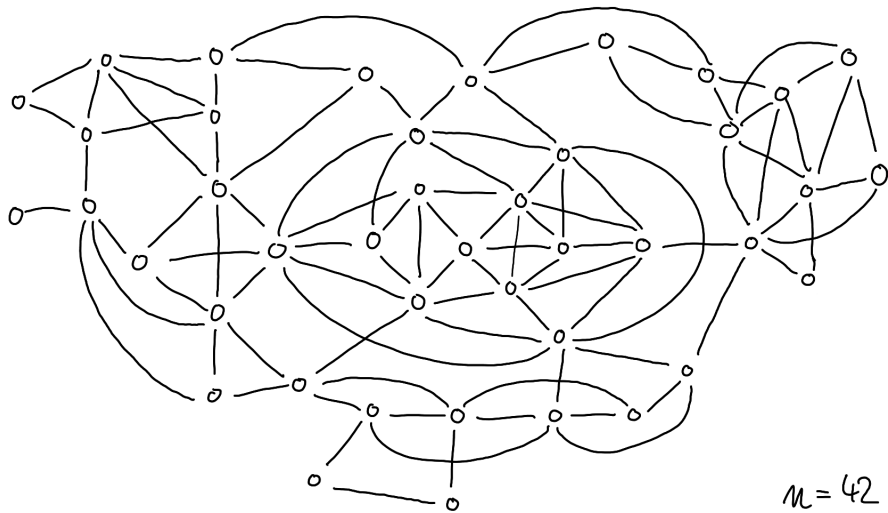
Often we can reduce the size of a problem by some easy preprocessing steps. For example:

- Before searching for a clique of size k , remove all nodes of degree $< k$ from the graph.
- Before checking a logical formula in CNF, remove all trivial clauses from the formula (next week!).

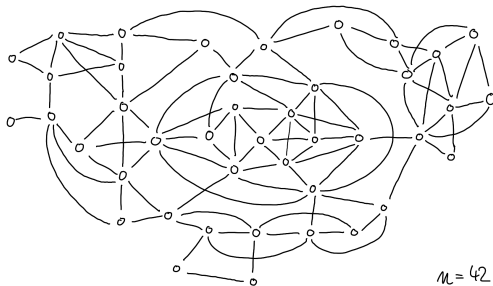
The resulting reduced problem is often called the *kernel*, it contains the “*hard part*” of the problem. It is equivalent to the original problem (in the sense that a solution for the original problem can be constructed from a solution of the kernel in polynomial time).

In the literature, the definition of kernels requires that the kernel is of size $\mathcal{O}(f(k))$, therefore only problems in the class FPT can have kernels. However, we can apply the techniques also on other problems (see examples above).

Example: 5-Clique



Example: 5-Clique



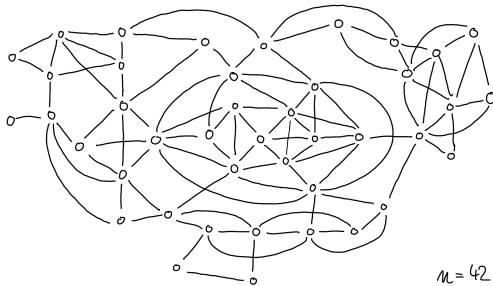
$$n = 42$$

Does this graph contain a clique of size 5?

Brute-force approach: check all subsets of size 5: $\binom{42}{5} = 850668$.

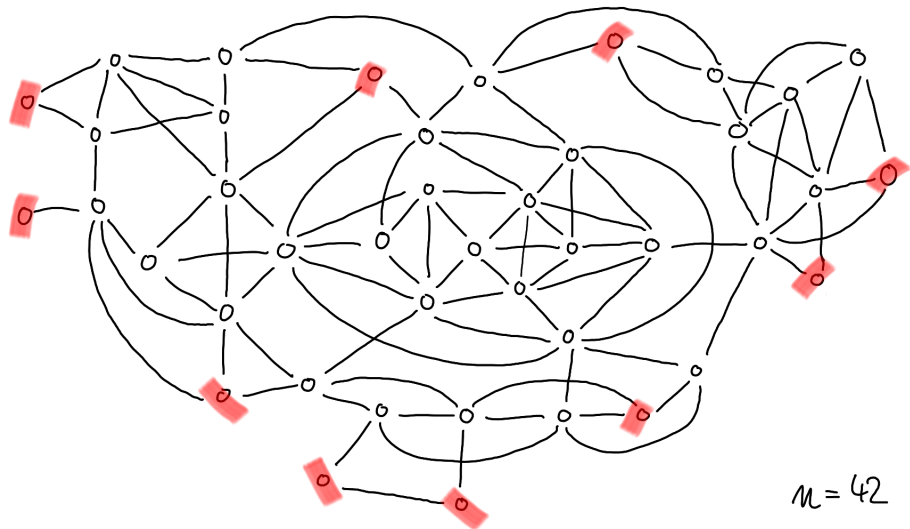
But we know that every node in a clique of size 5 has to have degree ≥ 4 . This allows us to simplify the graph first.

Example: 5-Clique

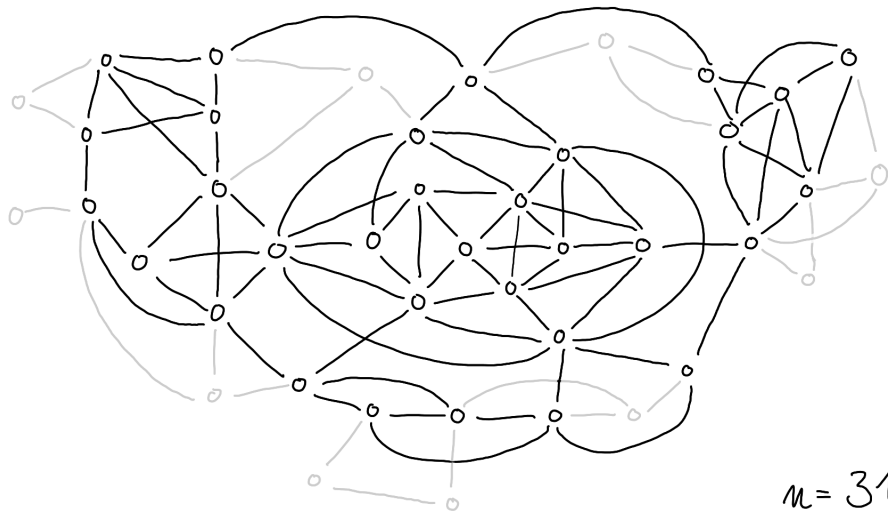


We build a kernel-graph: iteratively remove all nodes of degree < 4 , since these will not be part of a 5-clique.

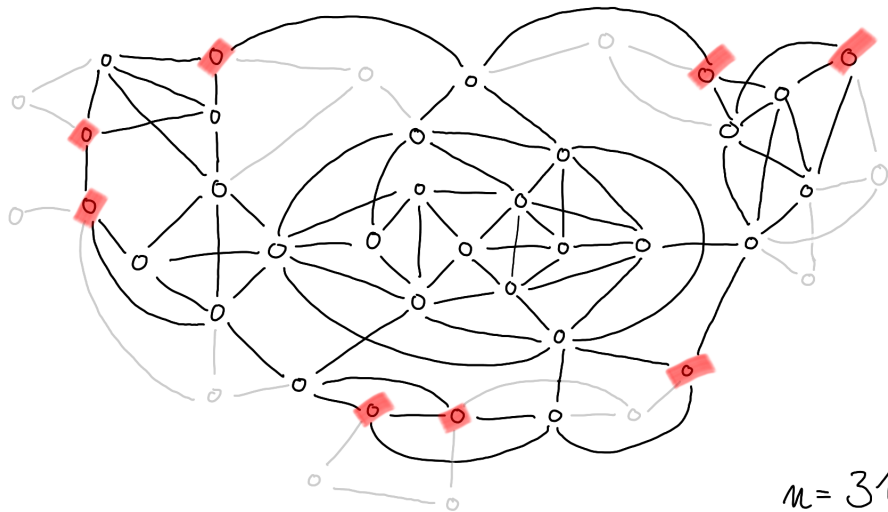
Example: 5-Clique: kernel step 1: all nodes of degree < 4



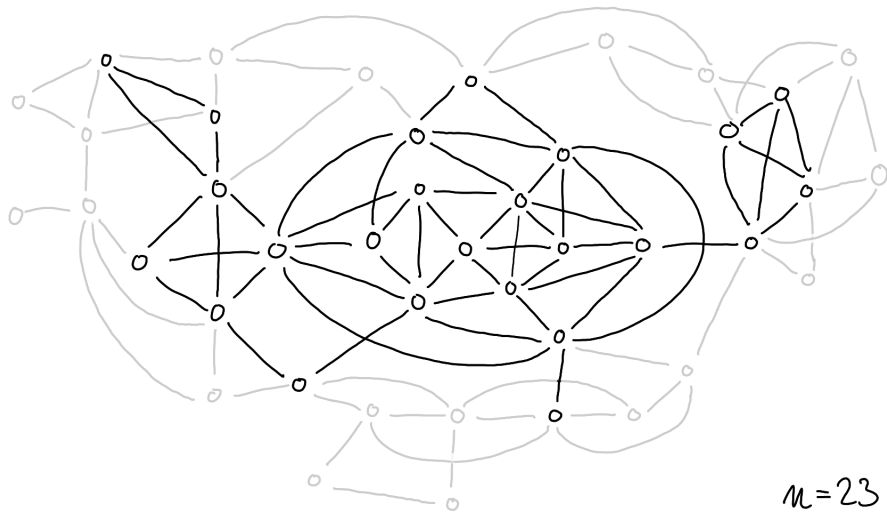
Example: 5-Clique



Example: 5-Clique: kernel step 2: all nodes of degree < 4

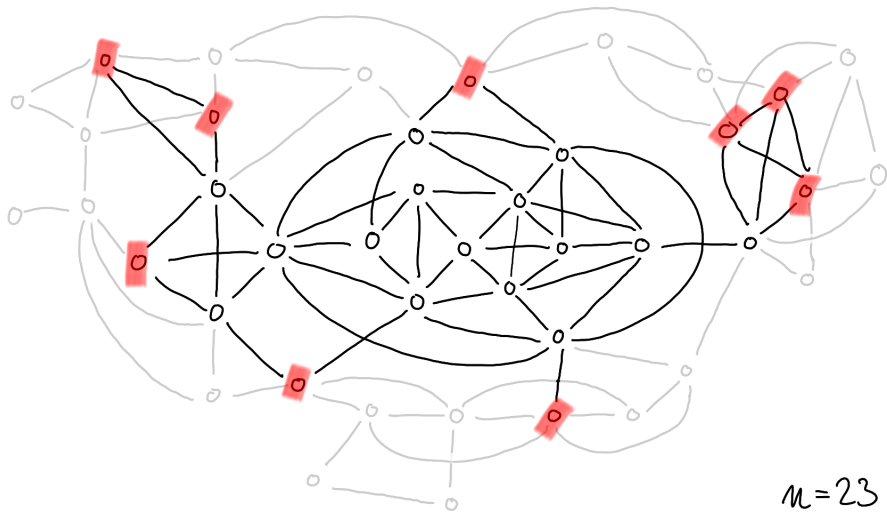


Example: 5-Clique

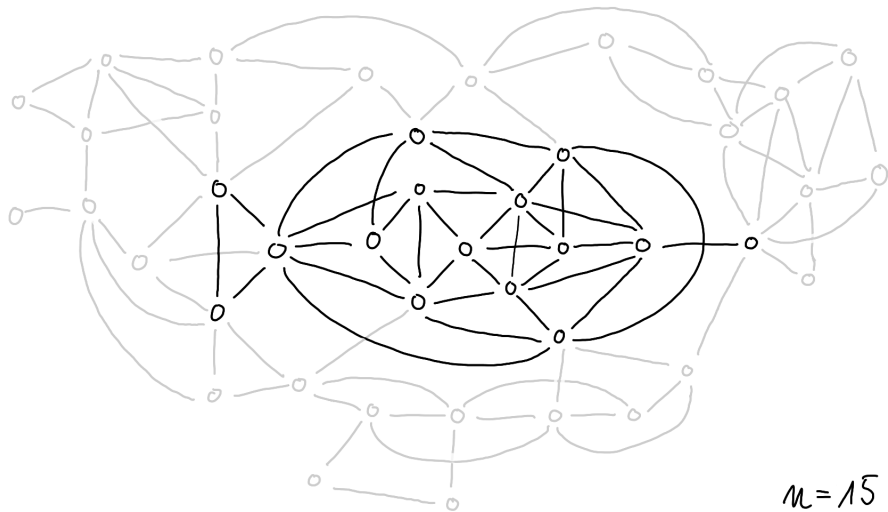


$$n=23$$

Example: 5-Clique: kernel step 3: all nodes of degree < 4

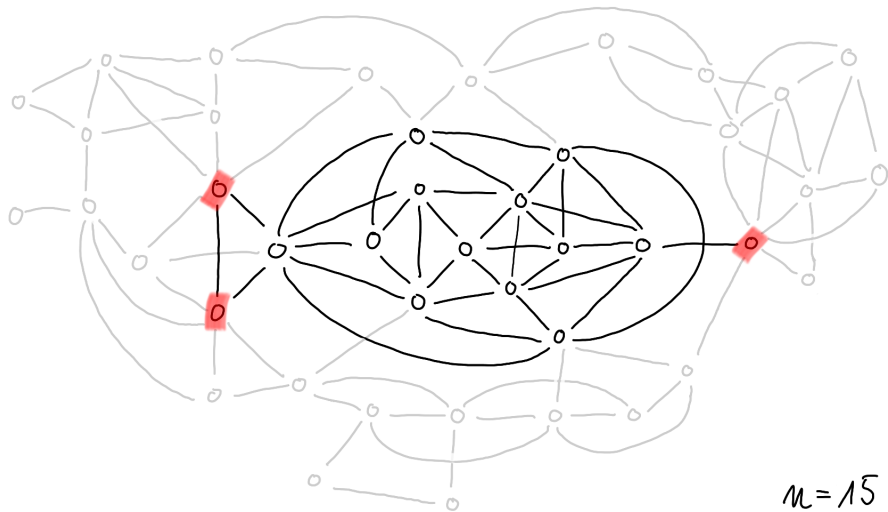


Example: 5-Clique

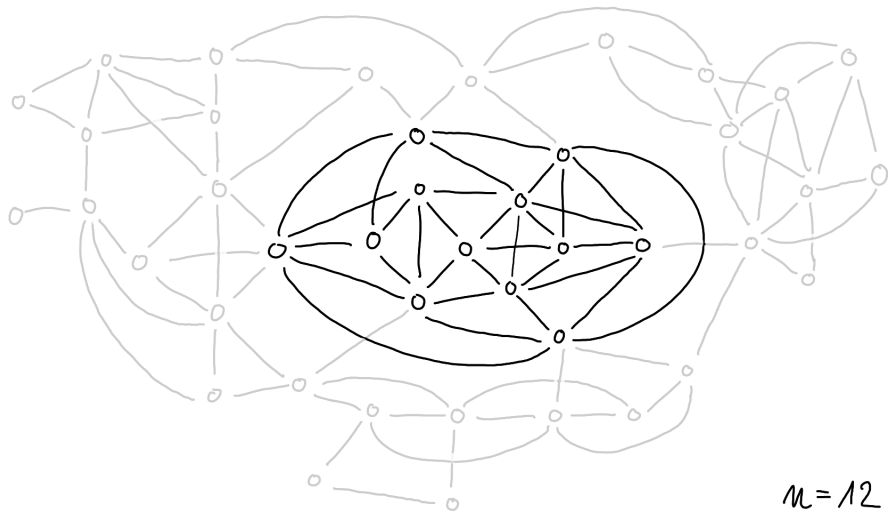


$$n=15$$

Example: 5-Clique: kernel step 4: all nodes of degree < 4

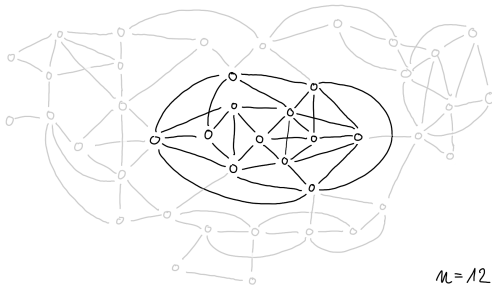


Example: 5-Clique



$$n=12$$

Example: 5-Clique



Now *only 12 nodes remain*, we have to check every subset of size 5 in this graph: $\binom{12}{5} = 792$ subsets to check (in contrast to > 850000 in the full graph).

Further optimization: Check for cliques of *size 4*, because only nodes in a 4-clique can be part of a 5-clique. There are only $\binom{12}{4} = 495$ subsets to check.

Exercises

Exercise

- Problem 1773-A from codeforces.com (<https://codeforces.com/problemset/problem/1773/A>). This is a Problem from the Northern Eurasia Finals of the latest ICPC Competition (2022–2023).
- Design at least three additional test cases.
- Add your code to your repository.

The Problem setting is also included on the next page.

Problem A. Amazing Trick

Time limit: 3 seconds
Memory limit: 1024 megabytes

Alice is a magician and she creates a new trick. She has n cards with different numbers from 1 to n written on them. First, she asks an audience member to shuffle the deck and put cards in a row. Let's say the i -th card from the left has the number a_i on it.

Then Alice picks two permutations p and q . There is a restriction on p and q — **permutations can't have fixed points**. Which means $\forall i : p_i \neq i$ and $q_i \neq i$.

After permutations are chosen, Alice shuffles the cards according to them. Now the i -th card from the left is the card $a[p[q[i]]]$. The trick is considered successful if i -th card from the left has the number i on it after the shuffles.

Help Alice pick the permutations p and q or say it is not possible for the specific starting permutation a .

Input

The first line of the input contains the number of tests t ($1 \leq t \leq 10^5$).

Each test is described in two lines. The first line contains one integer n — the number of cards ($1 \leq n \leq 10^5$). The second line contains n integers a_i — the initial permutation of the cards ($1 \leq a_i \leq n$; $\forall i \neq j : a_i \neq a_j$).

It is guaranteed that the sum of n over all tests does not exceed 10^5 .

Output

Print the answer for each test case in the same order the cases appear in the input.

For each test case, print “Impossible” in a single line, if no solution exists.

Otherwise, print “Possible” in the first line, and in the following two lines print permutations p and q .

Example

standard input	standard output
4	Impossible
2	Possible
2 1	3 1 2
3	2 3 1
1 2 3	Possible
4	3 4 2 1
2 1 4 3	3 4 2 1
5	Possible
5 1 4 2 3	4 1 2 5 3
	3 1 4 5 2

Exam Questions

- Describe a kernel of the k -VertexCover-Problem: figure out which nodes have to (and which don't have to) be in the VertexCover.
- Design an algorithm (pseudo-code) that constructs a minimal vertex cover for graphs where all nodes have degree at most 2. Your algorithm should run in time $\mathcal{O}(n^c)$ where n is the number of nodes in the graph, and c is a constant (bonus point if you achieve $c = 1$). Prove the correctness and the runtime of your algorithm.
- Describe your approach at this weeks programming exercise (Problem 1773-A). How does your algorithm find a pair of valid permutations?